Brute-Force KNN Classifier Assignment

Instructor: Your Name

September 28, 2025

Goal

Implement a **brute-force K-Nearest Neighbors (KNN)** classifier *from scratch* to classify customers into service categories. Do not use `sklearn`'s KNN implementation for prediction.

Dataset (Embedded Version)

Below is a sample of the `Telecustomers.csv` dataset (15 rows). Save it as a CSV file before starting:

```
region,tenure,age,marital,address,income,ed,employ,retire,gender,reside,custcat
2,13,44,1,9,64.0,4,5,0,0,2,1
1,34,33,1,2,136.0,5,13,0,0,1,4
2,58,52,1,24,116.0,1,29,0,0,1,3
1,33,33,0,14,33.0,2,0,0,1,1,2
2,23,30,1,9,30.0,1,2,0,0,4,3
1,45,25,0,7,86.0,4,4,0,0,1,2
2,12,47,1,4,54.0,3,10,0,0,2,1
2,67,50,1,23,120.0,2,33,0,0,1,4
1,35,28,0,11,40.0,3,6,0,0,2,2
2,27,46,1,16,95.0,4,20,0,0,1,3
1,15,31,0,6,65.0,3,5,0,0,1,2
2,48,36,1,21,110.0,2,18,0,0,2,4
1,25,29,0,10,50.0,1,3,0,0,1,2
2,40,54,1,19,99.0,5,25,0,0,2,3
1,30,38,1,12,70.0,4,8,0,0,1,1
```

The target column is `custcat` with values:

- 1 = Basic Service
- 2 = E-Service
- 3 = Plus Service
- 4 = Total Service

Assignment Tasks

1. Load and inspect the dataset: show head, dtypes, missing values, and class distribution.
2. Preprocess:

- Encode categorical variables if needed.
- Standardize features (z-score).
- Train/test split (80/20).

3. Implement brute-force KNN with:
   - Euclidean distance
   - Majority voting
   - Tie-breaking rule (document it)
4. Plot accuracy and error vs. k (for $1 \leq k \leq 40$).
5. Identify best k, report test accuracy, and show confusion matrix.
6. Measure runtime for predictions at $k = 5$. Relate to complexity $O(m \cdot n \cdot d)$.

Hints

- Standardize numerical features, otherwise income dominates distances.
- Use broadcasting in NumPy for efficient distance computation.
- Use cross-validation on training data to pick k.
- Mention the complexity tradeoff: brute-force vs. tree/ANN methods.

Starter Snippet

```
import numpy as np

def pairwise_distances(X_test, X_train):
    xt2 = np.sum(X_test**2, axis=1, keepdims=True)
    xr2 = np.sum(X_train**2, axis=1, keepdims=True).T
    cross = X_test @ X_train.T
    d2 = xt2 + xr2 - 2*cross
    return np.sqrt(np.maximum(d2, 0.0))

def knn_predict(X_train, y_train, X_test, k=5):
    dists = pairwise_distances(X_test, X_train)
    idx = np.argsort(dists, axis=1)[:, :k]
    neigh = y_train[idx]
    preds = []
    for row in neigh:
        vals, counts = np.unique(row, return_counts=True)
        preds.append(vals[np.argmax(counts)])
    return np.array(preds)
```

Rubric (100 pts)

Item	Points
Preprocessing & exploration	15
Correct brute-force KNN	25
Plots (accuracy/error vs. k)	15
Confusion matrix & analysis	15
Timing & complexity reasoning	10
Code quality & report clarity	20
Total	100